

MoSCoW REQUIREMENT PRIORITISATION

BonVoyage: An AI-Assisted Collaborative Trip Planner

Course: Software Engineering

Department of Computer Science and Engineering, BML Munjal University

1. OBJECTIVE

The objective of this exercise is to identify the principal requirements of the BonVoyage collaborative trip planner and to prioritise them using the MoSCoW technique, so that the order in which the system is built reflects the order in which its parts create value. The requirement set covers functional, non-functional, user and system requirements, and each entry is accompanied by a single-line justification for the priority it has been assigned.

2. BASIS OF PRIORITISATION

The requirements below are drawn from the approved project synopsis and the software requirements specification prepared for BonVoyage. The four MoSCoW categories have been applied with the following interpretation, which is the conventional reading of the technique:

- **Must Have:** the release has no viable value without it. If the item is dropped, the system cannot perform the task it exists to perform.
- **Should Have:** important and painful to omit, but the release still delivers its core value if the item is deferred to a later iteration.
- **Could Have:** desirable, and included only if time and effort permit. These items form the contingency that is surrendered first when the schedule tightens.
- **Won't Have:** consciously excluded from the current version, recorded so that the exclusion is a decision rather than an oversight.

A deliberate distinction is drawn between capability and convenience. A requirement is treated as a Must Have only where its absence would break the planning workflow, not merely degrade it. On this reading, the enforcement of hard constraints is essential while the generation of a day-wise itinerary is not, because a group can still reach a decision from a ranked shortlist and a set of costed packages.

3. IDENTIFIED REQUIREMENTS AND THEIR PRIORITISATION

Table 1 lists the twenty requirements identified for the system, together with their type, assigned priority and justification.

Table 1: MoSCoW prioritisation of BonVoyage requirements

Req. ID	Requirement	Type	Priority	Justification
R1	The system shall allow a user to register, authenticate and maintain a secure session.	Functional	Must Have	No preference can be attributed to a member who cannot be identified.
R2	The system shall allow a user to create a trip, invite members and manage the group roster.	Functional	Must Have	The trip group is the context on which every later function depends.
R3	The system shall collect each member's destination preferences, activity interests, budget range and availability window through a structured form.	Functional	Must Have	Structured input is the only comparable data the aggregation stage can operate on.
R4	The system shall aggregate individual submissions into a group model, compute the common availability window and report constraints that cannot be satisfied together.	Functional	Must Have	Without a shared travel window no recommendation can be feasible.
R5	The system shall eliminate destinations that violate the total budget, trip duration or travel radius before any ranking is performed.	Functional	Must Have	An infeasible destination is not a weaker option but no option at all.
R6	The system shall score the remaining destinations by group fit and present a ranked shortlist.	Functional	Must Have	Producing a defensible group compromise is the central purpose of the system.
R7	The system shall state, for each recommendation, which member preferences it satisfies and which it does not.	Functional	Must Have	A group will not accept a compromise whose reasoning it cannot inspect.
R8	The system shall generate multiple trip packages, each with an itemised cost breakdown shown as a group total and a per-person figure.	Functional	Must Have	Comparable costed options are what the group ultimately chooses between.
R9	The trip organiser shall be able to set the group-level parameters of origin, trip duration, total budget ceiling and travel radius.	User	Must Have	These parameters define the feasible space within which all filtering occurs.
R10	Preference and travel-history data shall be visible only to the owning member and, in aggregated form, to that member's own trip group, and credentials shall be stored as salted hashes.	Non-functional	Must Have	Personal data carries an obligation that is independent of feature scope.
R11	The system shall persist users, trips, preferences, packages and itineraries in a relational store with enforced referential integrity.	System	Must Have	Recommendations cannot be revisited or audited without reliable persistence.

Req. ID	Requirement	Type	Priority	Justification
R12	The system shall generate a day-wise itinerary for the selected package using a language model supplied with verified structured data.	Functional	Should Have	It completes the end-to-end plan but is scheduled last and depends on external data, so the release must remain viable without it.
R13	Any member shall be able to accept, modify or reject a generated recommendation or itinerary.	User	Should Have	Generated output must remain correctable, so this accompanies the generation requirement.
R14	The system shall fall back to cached or curated data when an external provider is unavailable, labelling each figure by its provenance.	Non-functional	Should Have	External services are unreliable, and an unlabelled estimate would mislead the user.
R15	Interactive screens shall respond within three seconds and a destination shortlist shall be produced within ten seconds under normal load.	Non-functional	Should Have	Slow response discourages members from completing their submissions.
R16	External map, routing, accommodation and language-model services shall be accessed through a single mediating module with caching and quota control.	System	Should Have	The core flow can operate on the curated dataset alone, so live integration improves rather than enables it.
R17	The system shall apply a fairness adjustment across successive recommendation rounds so that the same member is not repeatedly under-served.	Non-functional	Could Have	It improves perceived fairness but the scoring function remains valid without it.
R18	The system shall allow members to vote on the shortlisted destinations before a package is selected.	User	Could Have	Voting aids consensus, although the ranked shortlist already produces a defensible choice.
R19	The system shall provide online payment and booking confirmation for the selected package.	Functional	Won't Have	The product is decision support, and bookings are completed on external platforms.
R20	The system shall display real-time seat and room availability across all Indian transport and accommodation operators.	System	Won't Have	Nationwide live inventory is not uniformly accessible within the scope of this version.

4. DISTRIBUTION OF ASSIGNED PRIORITIES

Table 2 summarises how the requirements are distributed across the four categories.

Table 2: Distribution of requirements across MoSCoW categories

Priority	Count	Share	Character of the items in this category
Must Have	11	55 %	Identity, group formation, preference capture, constraint enforcement, ranking, explanation, costing, persistence and data protection
Should Have	5	25 %	Itinerary generation, human validation of generated output, graceful degradation, response time and live data integration
Could Have	2	10 %	Fairness adjustment across recommendation rounds and group voting on the shortlist
Won't Have	2	10 %	Payment and booking confirmation, and nationwide real-time inventory

Two observations follow from this distribution. First, Must Have items account for slightly over half of the requirement set, which sits within the convention that they should not exceed roughly sixty per cent of a planned release. The requirements in that group form a connected chain rather than a list of independent features, since preference capture without constraint filtering, or ranking without explanation, would leave the system unable to complete the task it exists for.

Second, the Could Have group is small. Because Could Have items are the contingency that a team surrenders first when a schedule tightens, a set of only two such items leaves little room for manoeuvre. This is consistent with the constraint already recorded in the project documentation, namely that the core feature set is ambitious relative to a single academic semester. The practical consequence is that if development time runs short, the reduction will have to come from the Should Have group, and the most likely candidates are live data integration, for which a curated fallback dataset already exists, and the generated itinerary, which a group can assemble manually once a package has been chosen.

5. CONCLUDING REMARKS

The prioritisation above defines a minimum viable release consisting of the eleven Must Have requirements, which together allow a group to form, submit preferences, receive a feasible ranked shortlist with an explanation, and compare costed packages. Every subsequent category extends that release without altering its structure, which means the schedule can be shortened at the margin without redesigning the system.